

Robotaxi 2026 — Behavior Tree Kapsamlı Proje Rehberi

Bu doküman projemizde ne yaptığımızı, yazdığımız kodları, kullandığımız mimariyi ve her bir parçanın ne işe yaradığını **en temelden** anlatan kapsamlı bir rehberdir.

1. BEHAVIOR TREE (BT) NEDİR?

Behavior Tree, robotun **karar mekanizmasıdır**. "Şimdi ne yapmalıyım?" sorusuna cevap verir.

Gerçek hayat benzetmesi: Bir şoför düşünün:

- Önce yol açık mı kontrol eder → açıksa sürer
- Kırmızı ışık varsa durur → yeşil olunca devam eder
- Yaya varsa durur → geçince devam eder

BT de aynı mantığı **ağaç yapısında** kodlar. Her karar bir "node" (düğüm) olur.

Neden BT? (if/else değil?)

```
// if/else ile yazsan:
if (acil_durum) { dur(); }
else if (yaya_var) { dur(); bekle(); }
else if (kirmizi_isik) { dur(); bekle(); }
else if (kavsakta) { yavasla(); don(); }
else { normal_sur(); }
// → 50+ senaryo olunca SPAGETTI KOD olur, bakımı imkansız
```

BT'de her senaryo **bağımsız bir node**. Ekle/çıkar/değiştir — diğerlerini etkilemez.

BT'nin 3 Temel Dönüş Değeri

Her node çalışınca şu 3 değerden birini döner:

Değer	Anlamı	Örnek
SUCCESS	"İşim bitti, başarılı"	Hız ayarlandı ✓
FAILURE	"İşim bitti, başarısız"	Yaya yok (tehlike yok)
RUNNING	"Henüz bitmedi, bekle"	15sn bekleme devam ediyor...

BT'nin 4 Kontrol Node'u

Bu node'lar çocuklarını (alt node'ları) nasıl çalıştıracağına karar verir:

Sequence (Sıralı): Çocukları sırayla çalıştırır. Biri FAILURE dönerse DURUR.

```
<Sequence>          <!-- Hepsi başarılı olmalı -->
  <YavasLa />        <!-- SUCCESS → sonrakine geç -->
  <DurCizgisindeKal /> <!-- SUCCESS → sonrakine geç -->
```

```
<YesilBekle />      <!-- FAILURE → Sequence FAILURE döner, hepsi durur -->
</Sequence>
```

Fallback (Yedek plan): Çocukları sırayla dener. Biri SUCCESS dönerse DURUR.

```
<Fallback>          <!-- İlk başarılı olan yeter -->
  <SeritDegistir />  <!-- FAILURE → sonrakini dene -->
  <RotaDegistir />   <!-- SUCCESS → Fallback SUCCESS döner -->
</Fallback>
```

ReactiveSequence: Sequence gibi ama her tick'te BAŞTAN kontrol eder (güvenlik için).

Repeat: Çocuğunu N kez tekrarlar.

2. NODE TÜRLERİ — C++ SINIF HİYERARŞİSİ

BT.CPP v3 kütüphanesinde 3 ana node tipi vardır:

2.1 ConditionNode — "Koşul Kontrol"

- **Tek iş:** Bir şeyi kontrol et, SUCCESS veya FAILURE dön
- **RUNNING dönemez!** (anlık kontrol)
- **Yan etkisi yoktur** (hiçbir şeyi değiştirmez)

```
// HPP dosyası (sınıf tanımı):
class HasMoreSegments : public BT::ConditionNode { // ← ConditionNode'dan türetildi
public:
  // Constructor – her node'da AYNI şablon
  HasMoreSegments(const std::string& name, const BT::NodeConfiguration& config)
    : BT::ConditionNode(name, config) {}

  // Port tanımı – bu node hangi verileri alıyor/veriyor
  static BT::PortsList providedPorts() {
    return {
      BT::InputPort<int>("seg_index"), // Blackboard'dan OKU
      BT::InputPort<int>("route_size") // Blackboard'dan OKU
    };
  }

  // tick() – node çalışınca çağrılan FONKSİYON
  BT::NodeStatus tick() override;
};

// CPP dosyası (implementasyon):
BT::NodeStatus HasMoreSegments::tick()
{
  int seg_index = 0, route_size = 0;
  getInput("seg_index", seg_index); // Port'tan oku
  getInput("route_size", route_size); // Port'tan oku

  return (seg_index < route_size)
```

```

    ? BT::NodeStatus::SUCCESS    // Daha segment var
    : BT::NodeStatus::FAILURE;    // Bitti
}

```

2.2 SyncActionNode — "Anlık Eylem"

- **Tek iş:** Bir eylem yap ve hemen bitir
- SUCCESS veya FAILURE döner (RUNNING dönemez)
- **Yan etkisi VARDIR** (bir şeyi değiştirir: topic publish, blackboard yaz)

```

// HPP:
class SetMaxSpeed : public BT::SyncActionNode { // ← SyncActionNode'dan türetildi
public:
    SetMaxSpeed(const std::string& name, const BT::NodeConfiguration& config)
        : BT::SyncActionNode(name, config) {}

    static BT::PortsList providedPorts() {
        return { BT::InputPort<double>("speed") };
    }

    BT::NodeStatus tick() override;
};

// CPP:
BT::NodeStatus SetMaxSpeed::tick()
{
    double speed = 0.0;
    if (!getInput("speed", speed)) { // Port'tan oku, başarısızsa FAILURE
        return BT::NodeStatus::FAILURE;
    }

    // Blackboard'a kaydet (BT içi paylaşım)
    config().blackboard->set("current_max_speed", speed);

    // ROS2 topic'e publish et (BT dışı paylaşım)
    auto ros = getRosNode(config()); // Blackboard'dan ROS node al
    if (ros) {
        auto pub = ros->create_publisher<std_msgs::msg::Float64>("/vehicle/max_speed",
10);
        std_msgs::msg::Float64 msg;
        msg.data = speed;
        pub->publish(msg);
    }

    return BT::NodeStatus::SUCCESS; // İş bitti
}

```

2.3 StatefulActionNode — "Süreçli Eylem"

- **Birden fazla tick sürer** (RUNNING dönebilir)
- 3 fonksiyon: `onStart()`, `onRunning()`, `onHalted()`

```

// HPP:
class Dwell : public BT::StatefulActionNode { // ← StatefulActionNode
public:
    Dwell(const std::string& name, const BT::NodeConfiguration& config)
        : BT::StatefulActionNode(name, config) {}

    static BT::PortsList providedPorts() {
        return {
            BT::InputPort<double>("min_sec"),
            BT::InputPort<double>("max_sec")
        };
    }

    BT::NodeStatus onStart() override; // İlk tick'te çağrılır
    BT::NodeStatus onRunning() override; // Sonraki tick'lerde çağrılır
    void onHalted() override; // İptal edildiğinde çağrılır

private:
    std::chrono::steady_clock::time_point start_time_; // Başlangıç zamanı
    double wait_duration_sec_ = 15.0;
};

// CPP:
BT::NodeStatus Dwell::onStart() // ← Sadece İLK tick
{
    double min_sec = 15.0, max_sec = 20.0;
    getInput("min_sec", min_sec);
    getInput("max_sec", max_sec);
    wait_duration_sec_ = (min_sec + max_sec) / 2.0;
    start_time_ = std::chrono::steady_clock::now(); // Zamanlayıcı başlat
    return BT::NodeStatus::RUNNING; // "Henüz bitmedi"
}

BT::NodeStatus Dwell::onRunning() // ← Sonraki her tick
{
    auto elapsed = std::chrono::steady_clock::now() - start_time_;
    double sec = std::chrono::duration<double>(elapsed).count();

    if (sec >= wait_duration_sec_) {
        return BT::NodeStatus::SUCCESS; // Süre doldu, bitti
    }
    return BT::NodeStatus::RUNNING; // Henüz dolmadı, devam
}

void Dwell::onHalted() // ← Güvenlik refleksi tetiklenirse
{
    RCLCPP_WARN(btLogger(), "Dwell: bekleme kesildi");
}

```

3. PORT SİSTEMİ VE BLACKBOARD

Blackboard Nedir?

Tüm node'ların **ortak hafızası**. Bir node yazıyor, başka node okuyor.

```
Blackboard (ortak hafıza):
  seg_index = 3
  route_size = 12
  cruise_speed = 1.50
  light_color = "red"
  headlights_on = true
```

Port Tipleri

Port Tipi	Yön	Kullanım
InputPort<T>	Oku	Blackboard'dan veri AL
OutputPort<T>	Yaz	Blackboard'a veri YAZ
BidirectionalPort<T>	Oku+Yaz	Hem oku hem yaz (AdvanceSegment)

XML'de Port Bağlama

```
<!-- {süslü parantez} = Blackboard değişkenine bağla -->
<SetMaxSpeed speed="{cruise_speed}" />
<!-- ↑ cruise_speed Blackboard'dan okunur (1.50) -->

<!-- Sabit değer (süslü parantez yok) -->
<SetMaxSpeed speed="0.30" />
<!-- ↑ Doğrudan 0.30 kullanılır -->
```

4. ROS2 ALTYAPISI — bt_node_base.hpp

BT node'larımız ROS2 ile nasıl konuşuyor?

```
// bt_node_base.hpp – İki yardımcı fonksiyon

// 1) Logger – tüm node'lar aynı isimle log basar
inline rclcpp::Logger btLogger() {
  return rclcpp::get_logger("segment_bt");
}
// Kullanım: RCLCPP_INFO(btLogger(), "Hız: %.2f", speed);

// 2) ROS Node erişimi – publisher/subscriber oluşturmak için
inline rclcpp::Node::SharedPtr getRosNode(const BT::NodeConfiguration& config) {
  rclcpp::Node::SharedPtr node;
  config.blackboard->get("ros_node", node); // main.cpp'de set edilmişti
  return node;
}
```

```

}
// Kullanım:
// auto ros = getRosNode(config());
// auto pub = ros->create_publisher<std_msgs::msg::Bool>("/topic", 10);

```

main.cpp — Her şeyi birleştiren dosya

```

int main(int argc, char **argv) {
    rclcpp::init(argc, argv); // ROS2 başlat
    auto ros_node = rclcpp::Node::make_shared("bt_test"); // ROS node oluştur

    BT::BehaviorTreeFactory factory; // BT fabrikası
    robotaxi_bt::registerAllNodes(factory); // 43 node'u kaydet

    auto tree = factory.createTreeFromFile(xml_file); // XML'den ağaç oluştur
    tree.rootBlackboard()->set("ros_node", ros_node); // ROS node'u paylaş

    BT::NodeStatus result = tree.tickRoot(); // Ağacı çalıştır!
}

```

5. DOSYA YAPISI VE MODÜLER MİMARİ

HPP vs CPP Nedir?

- **HPP (Header):** Sınıf tanımı — "Bu node ne yapacak?" (arayüz)
- **CPP (Source):** Gerçek kod — "Nasıl yapacak?" (implementasyon)

```

src/robotaxi_bt/
├── include/robotaxi_bt/          ← HPP dosyaları (tanımlar)
│   ├── bt_node_base.hpp         ← ROS2 erişim altyapısı
│   ├── segment_loop_nodes.hpp  ← ✅ 5 node tanımı
│   ├── vehicle_control_nodes.hpp ← ✅ 4 node tanımı
│   ├── traffic_logic_nodes.hpp  ← ✅ 4 node tanımı
│   ├── mission_utility_nodes.hpp ← ✅ 3 node tanımı
│   ├── stub_nodes.hpp          ← ⏳ 27 stub tanımı
│   └── register_all_nodes.hpp   ← 43 node'u kayıt eden fonksiyon
├── src/                          ← CPP dosyaları (kodlar)
│   ├── main.cpp                ← Programın giriş noktası
│   ├── segment_loop_nodes.cpp  ← ✅ 5 node implementasyonu
│   ├── vehicle_control_nodes.cpp ← ✅ 4 node implementasyonu
│   ├── traffic_logic_nodes.cpp  ← ✅ 4 node implementasyonu
│   ├── mission_utility_nodes.cpp ← ✅ 3 node implementasyonu
│   └── stub_nodes.cpp          ← ⏳ 27 stub implementasyonu
├── behaviour_trees/
│   └── segment_bt.xml           ← Ana BT ağacı (494 satır)
└── CMakeLists.txt              ← Derleme konfigürasyonu

```

6. TAMAMLANAN 16 NODE — GRUP GRUP DETAYLI ANALİZ

GRUP 1: Segment Döngüsü (segment_loop_nodes)

Bu grup, rota üzerindeki segmentler arasında döngü yapar.

HasMoreSegments — `seg_index < route_size` mi kontrol eder.

- Tip: `ConditionNode`
- Portlar: `seg_index` (oku), `route_size` (oku)
- SUCCESS = daha segment var, FAILURE = bitti

IsSegmentType — Aktif segmentin tipini karşılaştırır.

- Tip: `ConditionNode`
- Portlar: `seg_type` (oku), `expected` (oku)
- Mantık: `seg_type == expected` → SUCCESS
- XML'de switch/case gibi kullanılır

IsTrafficControlActive — $Tur \geq 2$ mi kontrolü.

- Tip: `ConditionNode`
- Port: `tour` (oku)
- Mantık: Tur 1'de trafik ışıkları sahada yok, bu yüzden algılama kapalı

AdvanceSegment — `seg_index++` yapar.

- Tip: `SyncActionNode`
- Port: `seg_index` (`BidirectionalPort` — hem oku hem yaz)

ClearHandledFlags — Latch bayraklarını sıfırlar.

- Tip: `SyncActionNode`
- Mantık: `handled_stop_sign`, `handled_traffic_light` vb. false yapar
- Neden: Aynı DUR tabelasında sonsuz döngüyü önler

GRUP 2: Araç Kontrolü (vehicle_control_nodes)

SetMaxSpeed — Hız limiti ayarlar.

- Tip: `SyncActionNode`
- Port: `speed` (oku, double)
- Yaptığı: Blackboard'a yazar + `/vehicle/max_speed` topic'ine publish eder

StopVehicle — Aracı durdurur.

- Tip: `SyncActionNode`
- Yaptığı: `/cmd_vel` topic'ine sıfır Twist mesajı gönderir

TurnHeadlights — Far aç/kapat.

- Tip: `SyncActionNode`
- Port: `state` (oku, "on" veya "off")
- Yaptığı: `/vehicle/headlights` topic'ine Bool publish eder

Dwell — Belirli süre bekler (15-20 sn).

- Tip: **StatefulActionNode** (birden fazla tick sürer!)
- Portlar: `min_sec`, `max_sec` (oku)
- Yaşam döngüsü: `onStart`→`RUNNING`→`onRunning`→`RUNNING`→...→`SUCCESS`

GRUP 3: Trafik Mantığı (traffic_logic_nodes)

IsLightRed — Işık rengi "red" mi?

- Tip: ConditionNode
- Port: `color` (oku)

IsRoadSignType — Levha tipi kontrolü.

- Tip: ConditionNode
- Portlar: `sign_type` , `expected`

StopAndProceed — DUR tabelası mantığı.

- Tip: SyncActionNode
- Latch mekanizması: `handled_stop_sign` bayrağı ile aynı segmentte tekrar tetiklenmez

LogUnknownSign — Bilinmeyen levhayı loglar.

- Tip: SyncActionNode

GRUP 4: Görev Yardımcıları (mission_utility_nodes)

RecordMissionPoint — Görev noktası sayacı artırır (+30 puan).

- Tip: SyncActionNode
- Blackboard'daki `mission_points_reached` sayacını artırır

RecordParkEntryReached — Park girişi kaydı (+20 puan).

- Tip: SyncActionNode

SignalPassengerEvent — Yolcu olayı bildirir.

- Tip: SyncActionNode
- `/mission/passenger_event` topic'ine "pickup"/"dropoff" publish eder

7. STUB NODE'LAR — NEDEN STUB?

Stub = İskelet kod. Derlenir, çalışır, ama gerçek veri kullanmaz.

Örnek — gerçek vs stub karşılaştırma:

```
// GERÇEK (tamamlanmış) node:
BT::NodeStatus SetMaxSpeed::tick() {
    double speed; getInput("speed", speed);
    auto pub = ros->create_publisher<Float64>("/vehicle/max_speed", 10);
    pub->publish(msg); // ← GERÇEKTEN hız komutu gönderiyor
    return SUCCESS;
}

// STUB node:
BT::NodeStatus PedestrianAhead::tick() {
    // TODO: /perception/pedestrian topic'ini subscribe et
    return FAILURE; // ← Her zaman "yaya yok" diyor (sahte)
}
```


27 Stub — 3 Kategori

Harita Bağımlı (5): LoadMission, GetCurrentSegment, ReplanRoute, CalculateLaneChange, FindParkingSlot → Beklenen: segment_map.yaml + GeoJSON dosyası

Sensör Bağımlı (12): EmergencyStopRequested, PedestrianAhead, DynamicObstacleAhead, StaticObstacleInLane, TrafficLightAhead, StopSignAhead, vb. → Beklenen: Algılama ekibinin YOLO/Lidar ROS topic'leri

Nav2/Hareket Bağımlı (10): FollowLaneSegment, WaitForClear, ExecuteParking, BackUp, Spin, vb. → Beklenen: Nav2 NavigateToPose action server

8. XML AĞACI — BÜYÜK RESİM

```
MainTree
├─ SetBlackboard (hız parametreleri)
├─ LoadMission (GeoJSON → rota)
├─ WaitForGoSignal (UMS-2 bekle)
├─ DriveWithRecovery
│   └─ DriveAllSegments
│       └─ Repeat(-1) → SegmentLoop
│           ├─ HasMoreSegments
│           ├─ GetCurrentSegment
│           ├─ ClearHandledFlags
│           └─ ExecuteSegment (tip bazlı yönlendirme)
│               ├─ LANE_FOLLOW → SafeDrive
│               ├─ INTERSECTION → yavaşla + SafeDrive
│               ├─ ROUNDABOUT → boşluk bekle + SafeDrive
│               ├─ TUNNEL → far aç + SafeDrive + far kapat
│               ├─ PASSENGER_STOP → dur + 15sn bekle + sinyal
│               └─ PARKING → slot bul + park et
│           └─ AdvanceSegment (seg_index++)
│       └─ Recovery (geri git + dön)
└─ FinishMission (dur)

SafeDrive (her segment tipinde ortak)
├─ SafetyReflexes (güvenlik katmanları)
│   ├─ K1: Acil durdurma (UMS-1)
│   ├─ K2: Yaya kontrolü
│   ├─ K3: Dinamik engel
│   ├─ K4: Statik engel
│   └─ [Tur≥2 kapısı]
│       ├─ K5: Trafik ışığı
│       ├─ K6: DUR tabelası
│       └─ K7: Yol levhaları
└─ FollowLaneSegment (hedef git)
```

9. TICK MEKANİZMASI — ADIM ADIM

`tree.tickRoot()` çağrıldığında ne olur?

Tick #1:

```
MainTree → Sequence başlar
SetBlackboard: cruise_speed=1.50 → SUCCESS
LoadMission: rota yükle → SUCCESS
WaitForGoSignal: onStart() → RUNNING ← BURADA DURUYOR
(RUNNING dönünce ağaç bekler)
```

Tick #2:

```
MainTree → Sequence devam eder
WaitForGoSignal: onRunning() → SUCCESS (sinyal geldi)
DriveWithRecovery → Fallback başlar
DriveAllSegments → Repeat başlar
  HasMoreSegments: 0 < 12 → SUCCESS
  GetCurrentSegment: index=0, tip=LANE_FOLLOW → SUCCESS
  ClearHandledFlags → SUCCESS
  ExecuteSegment → Fallback başlar
    IsSegmentType: LANE_FOLLOW == LANE_FOLLOW → SUCCESS
    Seg_LaneFollow → Sequence başlar
      SetMaxSpeed: 1.50 → SUCCESS
      SafeDrive → ReactiveSequence başlar
        SafetyReflexes: tüm katmanlar kontrol → SUCCESS (tehlike yok)
        FollowLaneSegment: onStart() → RUNNING ← BURADA DURUYOR
```

Tick #3:

```
... (her tick'te SafetyReflexes TEKRAR kontrol edilir – güvenlik!)
FollowLaneSegment: onRunning() → RUNNING (hedefe gidiliyor)
```

Tick #N:

```
FollowLaneSegment: onRunning() → SUCCESS (hedefe ulaşıldı!)
AdvanceSegment: seg_index = 1 → SUCCESS
(Repeat yeni iterasyona geçer: HasMoreSegments kontrol...)
```

10. REGISTER SİSTEMİ VE CMakeLists

Node Kayıt (register_all_nodes.hpp)

```
inline void registerAllNodes(BT::BehaviorTreeFactory& factory) {
    // Her C++ sınıfını bir XML ismiyle eşle:
    factory.registerNodeType<HasMoreSegments>("HasMoreSegments");
    //           ↑ C++ sınıf adı           ↑ XML'deki ID
    // XML'de <HasMoreSegments .../> yazınca bu sınıf çalışır
    // ... toplam 43 node kaydedilir
}
```

CMakeLists.txt — Derleme

```
# Kütüphane oluştur (5 cpp dosyası birleşir)
add_library(segment_bt_nodes SHARED
    src/segment_loop_nodes.cpp
```

```
src/vehicle_control_nodes.cpp
src/traffic_logic_nodes.cpp
src/mission_utility_nodes.cpp
src/stub_nodes.cpp
)
# ROS2 bağımlılıkları
ament_target_dependencies(segment_bt_nodes
  behaviortree_cpp_v3 rclcpp geometry_msgs std_msgs
)
# Çalıştırılabilir dosya
add_executable(bt_test src/main.cpp)
target_link_libraries(bt_test segment_bt_nodes)
```

Derleme: `colcon build --packages-select robotaxi_bt` Çalıştırma: `ros2 run robotaxi_bt bt_test segment_bt.xml`

11. PROJE DURUM ÖZETİ

Metrik	Değer
Toplam node sayısı	43
Tamamlanan (gerçek kod)	16 (%37)
Stub (iskelet)	27 (%63)
XML ağacı satır sayısı	494
HPP dosyası sayısı	7
CPP dosyası sayısı	6
Build durumu	✅ Başarılı, 0 hata

Bizim yaptığımız:

- ✅ BT XML ağacı tasarımı (tüm yarışma senaryoları)
- ✅ 16 node'un gerçek C++ kodu (hpp + cpp)
- ✅ 27 stub node iskeleti
- ✅ ROS2 publisher altyapısı
- ✅ Modüler dosya yapısı
- ✅ Build sistemi
- ✅ Dokümanlar

Entegrasyon için beklenen:

- Algılama ekibi: YOLO/Lidar topic'leri
- Harita ekibi: segment_map.yaml
- Nav2: NavigateToPose action server
- Nav2: NavigateToPose action server

12. İMPLEMENTE EDİLEN C++ KODLARININ DETAYLI MİMARİSİ VE ANALİZİ (SIFIRDAN ANLATIM)

Bu bölümde, projemizde şu ana kadar C++ ile yazılmış olan tüm Behavior Tree node'larını ve destekleyici veri yapılarını teker teker, en temel seviyeden başlayarak mimari analizleriyle birlikte ele alacağız. Amacımız, sıfırdan başlayan birinin bile kodun satır satır ne yaptığını anlamasıdır.

12.1. ORTAK ALTYAPI: `bt_node_base.hpp`

Behavior Tree node'larımızın ROS 2 ve loglama kütüphaneleriyle haberleşmesi için ortak bir arayüze ihtiyacı vardır. Bu ihtiyacı `bt_node_base.hpp` karşılar.

```
#ifndef BT_NODE_BASE_HPP
#define BT_NODE_BASE_HPP

#include "rclcpp/rclcpp.hpp"
#include "behaviortree_cpp_v3/action_node.h"
#include "behaviortree_cpp_v3/condition_node.h"
#include <string>

namespace robotaxi_bt {

// Ortak logger fonksiyonu
inline rclcpp::Logger btLogger()
{
    return rclcpp::get_logger("segment_bt");
}

// Blackboard'dan ROS Node'a güvenli erişim sağlayan fonksiyon
inline rclcpp::Node::SharedPtr getRosNode(const BT::NodeConfiguration& config)
{
    rclcpp::Node::SharedPtr node;
    config.blackboard->get("ros_node", node);
    if (!node) {
        RCLCPP_ERROR(btLogger(), "Blackboard'da 'ros_node' bulunamadı! main.cpp'de set edilmeli.");
    }
    return node;
}

} // namespace robotaxi_bt
#endif
```

Mimari Analiz ve Satır Satır Anlatım:

- `btLogger()`** : ROS 2'nin standart loglama mekanizmasını (`rclcpp::get_logger`) kullanarak BT'ye özel "segment_bt" adında bir logger üretir. Böylece tüm loglar terminalde bu isim altında düzenlice görünür.
- `getRosNode(...)`** : Behavior Tree düğümleri normalde ROS 2 düğümü değildir; sadece C++ sınıflarıdır. ROS 2 publisher/subscriber oluşturabilmeleri için `main.cpp` 'de oluşturulan ana

ROS node pointer'ına (`ros_node`) ihtiyaç duyarlar. Bu fonksiyon, düğümün konfigürasyonundan (Blackboard) " `ros_node` " anahtarıyla bu pointer'ı çeker.

3. **Güvenlik Kontrolü:** `if (!node)` bloğu, eğer Blackboard'da ROS node'u tanımlanmamışsa hata basarak geliştiriciyi uyarır.

12.2. SEGMENT DÖNGÜSÜ NODE'LARI (`segment_loop_nodes`)

Bu düğümler, harita üzerindeki segmentler (yol parçaları) arasında dönmeyi, şeritleri kontrol etmeyi ve segment durumlarını güncellemeyi sağlar. Harici bir sensör bağımlılığı yoktur, tamamen saf mantık içerirler.

1. HasMoreSegments (ConditionNode)

- **Görevi:** Rota üzerinde gidilecek başka segment kalıp kalmadığını kontrol eder.
- **Sınıf Yapısı (`segment_loop_nodes.hpp`):**

```
class HasMoreSegments : public BT::ConditionNode {
public:
    HasMoreSegments(const std::string& name, const BT::NodeConfiguration&
config)
        : BT::ConditionNode(name, config) {}

    static BT::PortsList providedPorts() {
        return {
            BT::InputPort<int>("seg_index", "Şu anki segment indeksi"),
            BT::InputPort<int>("route_size", "Toplam segment sayısı")
        };
    }
    BT::NodeStatus tick() override;
};
```

- **İmplementasyon (`segment_loop_nodes.cpp`):**

```
BT::NodeStatus HasMoreSegments::tick() {
    int seg_index = 0;
    int route_size = 0;
    getInput("seg_index", seg_index);
    getInput("route_size", route_size);

    bool has_more = (seg_index < route_size);
    return has_more ? BT::NodeStatus::SUCCESS : BT::NodeStatus::FAILURE;
}
```

- **Satır Satır Analiz:**

- `providedPorts()` içinde iki adet `InputPort` (`seg_index` ve `route_size`) tanımlanmıştır. Bu veriler Blackboard'dan okunur.
 - `tick()` fonksiyonunda `getInput` ile bu veriler yerel değişkenlere aktarılır.
 - Eğer mevcut indeks, toplam segment sayısından küçükse `SUCCESS` dönerek döngünün devam etmesini sağlar. Eşit veya büyükse `FAILURE` dönerek döngüyü sonlandırır.
-

2. IsSegmentType (ConditionNode)

- **Görevi:** Aktif segmentin tipinin, beklenen tip (örneğin kavşak, tünel, şerit takibi) ile eşleşip eşleşmediğini kontrol eder.
- **Sınıf Yapısı (segment_loop_nodes.hpp):**

```
class IsSegmentType : public BT::ConditionNode {
public:
    IsSegmentType(const std::string& name, const BT::NodeConfiguration& config)
        : BT::ConditionNode(name, config) {}

    static BT::PortsList providedPorts() {
        return {
            BT::InputPort<std::string>("seg_type", "Aktif segment tipi"),
            BT::InputPort<std::string>("expected", "Beklenen tip (LANE_FOLLOW,
INTERSECTION, ...)")
        };
    }
    BT::NodeStatus tick() override;
};
```

- **İmplementasyon (segment_loop_nodes.cpp):**

```
BT::NodeStatus IsSegmentType::tick() {
    std::string seg_type, expected;
    getInput("seg_type", seg_type);
    getInput("expected", expected);

    bool match = (seg_type == expected);
    return match ? BT::NodeStatus::SUCCESS : BT::NodeStatus::FAILURE;
}
```

- **Satır Satır Analiz:**

- XML'den gelen aktif şerit tipi (seg_type) ile hedeflenen tip (expected) getInput ile okunur.
- String karşılaştırması yapılarak durum eşleşiyorsa SUCCESS , eşleşmiyorsa FAILURE dönülür. Bu sayede Behavior Tree XML tarafında bir switch-case yapısı simüle edilir.

3. IsTrafficControlActive (ConditionNode)

- **Görevi:** Yarışma kuralları gereği Tur 1'de trafik levhaları sahada yoktur. Bu node tur numarasını kontrol ederek levha/ışık algılama mantığının çalışıp çalışmayacağını kontrol eder.
- **Sınıf Yapısı (segment_loop_nodes.hpp):**

```
class IsTrafficControlActive : public BT::ConditionNode {
public:
    IsTrafficControlActive(const std::string& name, const
BT::NodeConfiguration& config)
        : BT::ConditionNode(name, config) {}

    static BT::PortsList providedPorts() {
```

```

        return { BT::InputPort<int>("tour", "Tur numarası (1, 2 veya 3)") };
    }
    BT::NodeStatus tick() override;
};

```

- **İmplementasyon (segment_loop_nodes.cpp):**

```

BT::NodeStatus IsTrafficControlActive::tick() {
    int tour = 1;
    getInput("tour", tour);
    bool active = (tour >= 2);
    return active ? BT::NodeStatus::SUCCESS : BT::NodeStatus::FAILURE;
}

```

- **Satır Satır Analiz:**

- tour parametresi okunur. Tur 2 veya Tur 3 ise trafik levha/ışık kurallarının aktif olması için SUCCESS dönlür. Tur 1 ise FAILURE dönülerek güvenlik refleksi içindeki o katmanların atlanması sağlanır.

4. AdvanceSegment (SyncActionNode)

- **Görevi:** Segment indeksini 1 artırarak bir sonraki segmentin işlenmesini sağlar.
- **Sınıf Yapısı (segment_loop_nodes.hpp):**

```

class AdvanceSegment : public BT::SyncActionNode {
public:
    AdvanceSegment(const std::string& name, const BT::NodeConfiguration&
config)
        : BT::SyncActionNode(name, config) {}

    static BT::PortsList providedPorts() {
        return { BT::BidirectionalPort<int>("seg_index", "Artırılacak segment
indeksi") };
    }
    BT::NodeStatus tick() override;
};

```

- **İmplementasyon (segment_loop_nodes.cpp):**

```

BT::NodeStatus AdvanceSegment::tick() {
    int seg_index = 0;
    getInput("seg_index", seg_index);
    seg_index++;
    setOutput("seg_index", seg_index);

    RCLCPP_INFO(btLogger(), "AdvanceSegment: yeni index = %d", seg_index);
    return BT::NodeStatus::SUCCESS;
}

```

- **Satır Satır Analiz:**

- `BidirectionalPort` (çift yönlü port) kullanır. Bu port tipi, aynı Blackboard anahtarı üzerinden hem okuma hem yazma yetkisi tanır.
- Değer okunur (`getInput`), 1 artırılır ve aynı anahtara geri yazılır (`setOutput`).
Düğüm anlık çalıştığı için işlem sonunda `SUCCESS` döner.

5. ClearHandledFlags (SyncActionNode)

- **Görevi:** Düğümün bir segmentten diğerine geçerken, önceki segmentte işlenmiş olan levha ve ışık bayraklarını sıfırlamasını sağlar.
- **Sınıf Yapısı (`segment_loop_nodes.hpp`):**

```
class ClearHandledFlags : public BT::SyncActionNode {
public:
    ClearHandledFlags(const std::string& name, const BT::NodeConfiguration&
config)
        : BT::SyncActionNode(name, config) {}

    static BT::PortsList providedPorts() { return {}; }
    BT::NodeStatus tick() override;
};
```

- **İmplementasyon (`segment_loop_nodes.cpp`):**

```
BT::NodeStatus ClearHandledFlags::tick() {
    auto bb = config().blackboard;
    bb->set("handled_stop_sign", false);
    bb->set("handled_traffic_light", false);
    bb->set("handled_road_sign", false);
    bb->set("handled_pedestrian_crossing", false);

    RCLCPP_DEBUG(btLogger(), "ClearHandledFlags: tüm bayraklar sıfırlandı");
    return BT::NodeStatus::SUCCESS;
}
```

- **Satır Satır Analiz:**

- Port tanımlaması yoktur. Doğrudan `config().blackboard` pointer'ı alınır.
- Blackboard üzerinden tüm engelleme/işleme bayrakları (`handled_*`) `false` değerine çekilir. Bu işlem, robotun aynı dur levhasında veya trafik ışığında kilitlenip (latch) kalmasını engeller.

12.3. ARAÇ KONTROL NODE'LARI (`vehicle_control_nodes`)

Bu grup, aracın fiziksel eylemlerini tetikleyen ve ROS 2 topic'lerine mesaj yayınlayan düğümlerden oluşur.

6. SetMaxSpeed (SyncActionNode)

- **Görevi:** Aracın şerit tipine göre alabileceği maksimum hız sınırını ayarlar.
- **Sınıf Yapısı (`vehicle_control_nodes.hpp`):**


```

class SetMaxSpeed : public BT::SyncActionNode {
public:
    SetMaxSpeed(const std::string& name, const BT::NodeConfiguration& config)
        : BT::SyncActionNode(name, config) {}

    static BT::PortsList providedPorts() {
        return { BT::InputPort<double>("speed", "Hedef hız m/s cinsinden") };
    }
    BT::NodeStatus tick() override;
};

```

• **İmplementasyon (vehicle_control_nodes.cpp):**

```

BT::NodeStatus SetMaxSpeed::tick() {
    double speed = 0.0;
    if (!getInput("speed", speed)) {
        RCLCPP_ERROR(btLogger(), "SetMaxSpeed: 'speed' portu okunamadı!");
        return BT::NodeStatus::FAILURE;
    }

    config().blackboard->set("current_max_speed", speed);

    auto ros = getRosNode(config());
    if (ros) {
        auto pub = ros->create_publisher<std_msgs::msg::Float64>
("/vehicle/max_speed", 10);
        std_msgs::msg::Float64 msg;
        msg.data = speed;
        pub->publish(msg);
    }

    RCLCPP_INFO(btLogger(), "SetMaxSpeed: %.2f m/s ayarlandı", speed);
    return BT::NodeStatus::SUCCESS;
}

```

• **Satır Satır Analiz:**

- speed portu okunur. Hız limiti Blackboard'a "current_max_speed" anahtarıyla kaydedilir (diğer düğümlerin okuyabilmesi için).
- Ardından getRosNode ile alınan paylaşımlı ROS node'u kullanılarak /vehicle/max_speed topic'ine std_msgs::msg::Float64 tipinde hız limiti publish edilir. Robotun hız limit kontrolü bu sayede aktifleşir.

7. StopVehicle (SyncActionNode)

- **Görevi:** Aracı acil durumda veya tabelada durdurmak için hız komutunu sıfırlar.
- **Sınıf Yapısı (vehicle_control_nodes.hpp):**

```

class StopVehicle : public BT::SyncActionNode {
public:
    StopVehicle(const std::string& name, const BT::NodeConfiguration& config)

```

```

        : BT::SyncActionNode(name, config) {}

    static BT::PortsList providedPorts() { return {}; }
    BT::NodeStatus tick() override;
};

```

- **İmplementasyon (vehicle_control_nodes.cpp):**

```

BT::NodeStatus StopVehicle::tick() {
    auto ros = getRosNode(config());
    if (ros) {
        auto pub = ros->create_publisher<geometry_msgs::msg::Twist>("/cmd_vel",
10);
        geometry_msgs::msg::Twist zero_vel;
        pub->publish(zero_vel);
    }

    RCLCPP_INFO(btLogger(), "StopVehicle: /cmd_vel sıfırlandı");
    return BT::NodeStatus::SUCCESS;
}

```

- **Satır Satır Analiz:**

- ROS 2'nin standart hız topic'i olan /cmd_vel 'e geometry_msgs::msg::Twist tipinde bir yayıncı oluşturulur.
- C++'ta geometry_msgs::msg::Twist nesnesi ilklendirildiğinde tüm iç değişkenleri (linear/angular x,y,z) varsayılan olarak 0.0 değerini alır. Bu boş mesaj publish edilerek robotun tekerleklerine giden güç kesilir.

8. TurnHeadlights (SyncActionNode)

- **Görevi:** Tünel girişinde farları açar, tünel çıkışında farları kapatır.
- **Sınıf Yapısı (vehicle_control_nodes.hpp):**

```

class TurnHeadlights : public BT::SyncActionNode {
public:
    TurnHeadlights(const std::string& name, const BT::NodeConfiguration&
config)
        : BT::SyncActionNode(name, config) {}

    static BT::PortsList providedPorts() {
        return { BT::InputPort<std::string>("state", "on veya off") };
    }
    BT::NodeStatus tick() override;
};

```

- **İmplementasyon (vehicle_control_nodes.cpp):**

```

BT::NodeStatus TurnHeadlights::tick() {
    std::string state;
    if (!getInput("state", state)) {

```

```

    RCLCPP_ERROR(btLogger(), "TurnHeadlights: 'state' portu okunamadı!");
    return BT::NodeStatus::FAILURE;
}

bool lights_on = (state == "on" || state == "ON");
config().blackboard->set("headlights_on", lights_on);

auto ros = getRosNode(config());
if (ros) {
    auto pub = ros->create_publisher<std_msgs::msg::Bool>
("/vehicle/headlights", 10);
    std_msgs::msg::Bool msg;
    msg.data = lights_on;
    pub->publish(msg);
}

RCLCPP_INFO(btLogger(), "TurnHeadlights: farlar %s", lights_on ? "AÇILDI" :
"KAPANDI");
return BT::NodeStatus::SUCCESS;
}

```

- **Satır Satır Analiz:**

- XML'den gelen "on" veya "off" string durumu okunur.
- Mantıksal bool durumuna çevrilir ve /vehicle/headlights topic'ine std_msgs::msg::Bool olarak publish edilir. Tünel kurallarına uyum sağlanmış olur.

9. Dwell (StatefulActionNode)

- **Görevi:** Yolcu duraklarında 15-20 saniye boyunca güvenli bir şekilde beklemeyi sağlar. Zaman alan bir süreç olduğu için StatefulActionNode olarak tasarlanmıştır.
- **Sınıf Yapısı (vehicle_control_nodes.hpp):**

```

class Dwell : public BT::StatefulActionNode {
public:
    Dwell(const std::string& name, const BT::NodeConfiguration& config)
        : BT::StatefulActionNode(name, config) {}

    static BT::PortsList providedPorts() {
        return {
            BT::InputPort<double>("min_sec", "Minimum bekleme süresi (saniye)"),
            BT::InputPort<double>("max_sec", "Maksimum bekleme süresi (saniye)")
        };
    }

    BT::NodeStatus onStart() override;
    BT::NodeStatus onRunning() override;
    void onHalted() override;

private:
    std::chrono::steady_clock::time_point start_time_;

```

```
double wait_duration_sec_ = 15.0;
};
```

- **Implementasyon (vehicle_control_nodes.cpp):**

```
BT::NodeStatus Dwell::onStart() {
    double min_sec = 15.0, max_sec = 20.0;
    getInput("min_sec", min_sec);
    getInput("max_sec", max_sec);

    wait_duration_sec_ = (min_sec + max_sec) / 2.0;
    start_time_ = std::chrono::steady_clock::now();

    RCLCPP_INFO(btLogger(), "Dwell: %.0f sn bekleme başladı (min=%.0f, max=%.0f)",
                wait_duration_sec_, min_sec, max_sec);
    return BT::NodeStatus::RUNNING;
}

BT::NodeStatus Dwell::onRunning() {
    auto elapsed = std::chrono::steady_clock::now() - start_time_;
    double sec = std::chrono::duration<double>(elapsed).count();

    if (sec >= wait_duration_sec_) {
        RCLCPP_INFO(btLogger(), "Dwell: %.1f sn bekleme tamamlandı", sec);
        return BT::NodeStatus::SUCCESS;
    }
    return BT::NodeStatus::RUNNING;
}

void Dwell::onHalted() {
    RCLCPP_WARN(btLogger(), "Dwell: HALTED – bekleme kesildi");
}
```

- **Satır Satır Analiz:**

- `onStart()` ilk tick çağrıldığında çalışır. `min_sec` ve `max_sec` okunarak ortalaması alınır ve `start_time_` kaydedilerek zamanlayıcı başlatılır. Düğümün sürdüğünü belirtmek için `RUNNING` döner.
- `onRunning()` sonraki tick'lerde çağrılır. Geçen süre monotonik saat (`steady_clock`) ile karşılaştırılır. Süre dolduysa `SUCCESS` , dolmadıysa `RUNNING` dönülür.
- `onHalted()` eğer bu bekleme sırasında bir engel algılanır ve güvenlik refleksleri bekleme işlemini durdurursa (kesinti) çalışır ve log basar.

12.4. TRAFİK MANTIK NODE'LARI (traffic_logic_nodes)

Bu grup, trafik ışıklarının ve levhaların mantıksal kontrollerini gerçekleştirir.

10. IsLightRed (ConditionNode)

- **Görevi:** Algılanan trafik ışığının kırmızı olup olmadığını test eder.
- **Sınıf Yapısı (traffic_logic_nodes.hpp):**

```

class IsLightRed : public BT::ConditionNode {
public:
    IsLightRed(const std::string& name, const BT::NodeConfiguration& config)
        : BT::ConditionNode(name, config) {}

    static BT::PortsList providedPorts() {
        return { BT::InputPort<std::string>("color", "Işık rengi stringi") };
    }
    BT::NodeStatus tick() override;
};

```

- **İmplementasyon (traffic_logic_nodes.cpp):**

```

BT::NodeStatus IsLightRed::tick() {
    std::string color;
    getInput("color", color);
    bool is_red = (color == "red" || color == "RED" || color == "kirmizi");
    return is_red ? BT::NodeStatus::SUCCESS : BT::NodeStatus::FAILURE;
}

```

- **Satır Satır Analiz:**

- Algılama katmanından gelen color portu okunur. String değeri kırmızıya eşitse SUCCESS (kırmızı var, dur), değilse FAILURE (kırmızı ışık yok) dönülür.

11. IsRoadSignType (ConditionNode)

- **Görevi:** Algılanan yol levhasının tipini doğrular.
- **Sınıf Yapısı (traffic_logic_nodes.hpp):**

```

class IsRoadSignType : public BT::ConditionNode {
public:
    IsRoadSignType(const std::string& name, const BT::NodeConfiguration& config)
        : BT::ConditionNode(name, config) {}

    static BT::PortsList providedPorts() {
        return {
            BT::InputPort<std::string>("sign_type", "Algılanan levha tipi"),
            BT::InputPort<std::string>("expected", "Beklenen tip (LANE_MERGE, NO_ENTRY, ...)")
        };
    }
    BT::NodeStatus tick() override;
};

```

- **İmplementasyon (traffic_logic_nodes.cpp):**

```

BT::NodeStatus IsRoadSignType::tick() {
    std::string sign_type, expected;
    getInput("sign_type", sign_type);

```

```

    getInput("expected", expected);
    bool match = (sign_type == expected);
    return match ? BT::NodeStatus::SUCCESS : BT::NodeStatus::FAILURE;
}

```

- **Satır Satır Analiz:**

- Görüntü işleme veya haritadan gelen levha tipi ile beklenen levha tipi karşılaştırılır. Doğru eşleşme durumunda `SUCCESS` dönülerek ilgili tabela aksiyonu alt ağacına geçiş sağlanır.

12. StopAndProceed (SyncActionNode)

- **Görevi:** DUR (TT-2) levhası görüldüğünde aracın durmasını ve ardından yol kontrolü yapıp devam etmesini (latch mekanizmasıyla) sağlar.
- **Sınıf Yapısı (`traffic_logic_nodes.hpp`):**

```

class StopAndProceed : public BT::SyncActionNode {
public:
    StopAndProceed(const std::string& name, const BT::NodeConfiguration&
config)
        : BT::SyncActionNode(name, config) {}

    static BT::PortsList providedPorts() { return {}; }
    BT::NodeStatus tick() override;
};

```

- **İmplementasyon (`traffic_logic_nodes.cpp`):**

```

BT::NodeStatus StopAndProceed::tick() {
    auto bb = config().blackboard;
    bool handled = false;
    bb->get("handled_stop_sign", handled);

    if (handled) {
        RCLCPP_DEBUG(btLogger(), "StopAndProceed: zaten işlendi, atlanıyor");
        return BT::NodeStatus::SUCCESS;
    }

    bb->set("handled_stop_sign", true);
    RCLCPP_INFO(btLogger(), "StopAndProceed: DUR tabelası – duruldu, devam ediliyor");
    return BT::NodeStatus::SUCCESS;
}

```

- **Satır Satır Analiz:**

- Latch (kilit) bayrağı kontrol edilir. Eğer bu DUR levhası o segmentte zaten işlendiyse (`handled == true`), doğrudan `SUCCESS` dönerek aracın dur kalk kısır döngüsüne girmesi önlenir.
- İlk defa görülüyorsa bayrak `true` yapılır, log basılır ve araç bir kez durduktan sonra geçişine izin verilerek `SUCCESS` dönülür.

13. LogUnknownSign (SyncActionNode)

- **Görevi:** Sınıflandırılmamış veya tanımlanamayan trafik levhalarını debug loglarına basar.
- **Sınıf Yapısı (traffic_logic_nodes.hpp):**

```
class LogUnknownSign : public BT::SyncActionNode {
public:
    LogUnknownSign(const std::string& name, const BT::NodeConfiguration&
config)
        : BT::SyncActionNode(name, config) {}

    static BT::PortsList providedPorts() {
        return { BT::InputPort<std::string>("sign_type", "Tanımsız levha tipi")
};
    }
    BT::NodeStatus tick() override;
};
```

- **İmplementasyon (traffic_logic_nodes.cpp):**

```
BT::NodeStatus LogUnknownSign::tick() {
    std::string sign_type;
    getInput("sign_type", sign_type);
    RCLCPP_WARN(btLogger(), "LogUnknownSign: tanımsız levha = '%s'",
sign_type.c_str());
    return BT::NodeStatus::SUCCESS;
}
```

- **Satır Satır Analiz:**

- XML'de expected listesindekilere uymayan levhalar bu fallback yaprağına düşer.
- Levha tipi terminale uyarı (RCLCPP_WARN) olarak yazdırılır ve bir sonraki adıma geçiş için SUCCESS dönülür.

12.5. GÖREV YARDIMCI NODE'LARI (mission_utility_nodes)

Bu düğümler, yarışma puanı hesaplamasında kritik olan görev ve durak başarımlarını kaydeder ve ROS 2 hakem arayüzü topic'lerine veri gönderir.

14. RecordMissionPoint (SyncActionNode)

- **Görevi:** Robotun bir görev noktasına (waypoint) ulaştığını teyit eder ve Blackboard'daki görev noktası sayacını günceller (+30 puan).
- **Sınıf Yapısı (mission_utility_nodes.hpp):**

```
class RecordMissionPoint : public BT::SyncActionNode {
public:
    RecordMissionPoint(const std::string& name, const BT::NodeConfiguration&
config)
        : BT::SyncActionNode(name, config) {}

    static BT::PortsList providedPorts() {
```

```

    return {
        BT::InputPort<std::string>("point", "Görev noktası koordinatı"),
        BT::InputPort<double>("tolerance", 1.0, "Tolerans (metre)")
    };
}
BT::NodeStatus tick() override;
};

```

- **İmplementasyon (mission_utility_nodes.cpp):**

```

BT::NodeStatus RecordMissionPoint::tick() {
    std::string point;
    double tolerance = 1.0;
    getInput("point", point);
    getInput("tolerance", tolerance);

    int count = 0;
    config().blackboard->get("mission_points_reached", count);
    config().blackboard->set("mission_points_reached", count + 1);

    RCLCPP_INFO(btLogger(), "RecordMissionPoint: nokta=%s, toplam=%d",
    point.c_str(), count + 1);
    return BT::NodeStatus::SUCCESS;
}

```

- **Satır Satır Analiz:**

- point (koordinat) ve tolerance değerleri okunur.
- Blackboard üzerindeki "mission_points_reached" sayacı 1 artırılır. İleride bu düğüm, /odom veya /amcl_pose topic'lerinden gelen anlık konum verisi ile hedef waypoint koordinatını karşılaştıracak şekilde güncellenecektir.

15. RecordParkEntryReached (SyncActionNode)

- **Görevi:** Park alanının başlangıç çizgisine ulaşıldığını Blackboard'a kaydeder (+20 puan).
- **Sınıf Yapısı (mission_utility_nodes.hpp):**

```

class RecordParkEntryReached : public BT::SyncActionNode {
public:
    RecordParkEntryReached(const std::string& name, const
    BT::NodeConfiguration& config)
        : BT::SyncActionNode(name, config) {}

    static BT::PortsList providedPorts() { return {}; }
    BT::NodeStatus tick() override;
};

```

- **İmplementasyon (mission_utility_nodes.cpp):**

```

BT::NodeStatus RecordParkEntryReached::tick() {
    config().blackboard->set("park_entry_reached", true);
}

```



```
RCLCPP_INFO(btLogger(), "RecordParkEntryReached: park giriři kaydedildi
(+20)");
return BT::NodeStatus::SUCCESS;
}
```

- **Satır Satır Analiz:**

- Blackboard üzerindeki "park_entry_reached" bayrağı true yapılarak park alanına giriş yapıldığı onaylanır.

16. SignalPassengerEvent (SyncActionNode)

- **Görevi:** Durakta yolcunun araca binmesi (pickup) veya inmesi (dropoff) durumlarını ROS 2 hakem sunucusuna ve ilgili düğümlere haber verir.
- **Sınıf Yapısı (mission_utility_nodes.hpp):**

```
class SignalPassengerEvent : public BT::SyncActionNode {
public:
    SignalPassengerEvent(const std::string& name, const BT::NodeConfiguration&
config)
        : BT::SyncActionNode(name, config) {}

    static BT::PortsList providedPorts() {
        return { BT::InputPort<std::string>("event_type", "pickup veya dropoff")
};
    }
    BT::NodeStatus tick() override;
};
```

- **İmplementasyon (mission_utility_nodes.cpp):**

```
BT::NodeStatus SignalPassengerEvent::tick() {
    std::string event_type;
    if (!getInput("event_type", event_type)) {
        RCLCPP_ERROR(btLogger(), "SignalPassengerEvent: 'event_type'
okunamadı!");
        return BT::NodeStatus::FAILURE;
    }

    config().blackboard->set("last_passenger_event", event_type);

    auto ros = getRosNode(config());
    if (ros) {
        auto pub = ros->create_publisher<std_msgs::msg::String>
("/mission/passenger_event", 10);
        std_msgs::msg::String msg;
        msg.data = event_type;
        pub->publish(msg);
    }

    RCLCPP_INFO(btLogger(), "SignalPassengerEvent: '%s' olayı yayınlandı",
event_type.c_str());
}
```

```
    return BT::NodeStatus::SUCCESS;
}
```

- **Satır Satır Analiz:**

- event_type ("pickup" veya "dropoff") okunarak Blackboard'a "last_passenger_event" olarak kaydedilir.
- ROS 2 üzerinden /mission/passenger_event topic'ine std_msgs::msg::String veri tipinde publish edilir. Bu durum hakem arayüzü tarafından skor tablosuna yansıtılır.

12.6. YOL BULUCU VE HARİTA MODELİ: segment_graph.hpp

Robotun yol bulma algoritmasını (Dijkstra) ve haritayı düğümler (GraphNode) ve yollar (Segment) şeklinde hafızaya almasını sağlayan C++ sınıfıdır. Harita bağımlı BT düğümlerinin (örneğin LoadMission , ReplanRoute) kalbidir.

```
#ifndef SEGMENT_GRAPH_HPP
#define SEGMENT_GRAPH_HPP

#include <string>
#include <vector>
#include <map>
#include <unordered_map>
#include <queue>
#include <limits>
#include <fstream>
#include <sstream>
#include <cmath>

namespace robotaxi_bt {

// Graf Düzümü: Koordinatlı harita noktaları (Kavşak, durak vb.)
struct GraphNode {
    std::string id;
    double x = 0.0;
    double y = 0.0;
};

// Segment (Kenar): İki düğüm arasındaki yönlü yol parçası
struct Segment {
    std::string id;
    std::string from_node; // Başlangıç düğümü ID'si
    std::string to_node;   // Bitiş düğümü ID'si
    std::string type;      // LANE_FOLLOW, INTERSECTION, ROUNDABOUT, TUNNEL, etc.
    std::string lane;      // Şerit (right/left)
    std::string meta;      // Ek veriler (yolcu olayı vb.)
    double cost = 1.0;     // Dijkstra ağırlığı (düğümler arası mesafe)

    // Bitiş hedef yönelimi
    double goal_x = 0.0;
    double goal_y = 0.0;
};

}
```

```

    double goal_yaw = 0.0;
};

// Planlanmış yol / Rota
struct Route {
    std::vector<Segment> segments;
    double total_cost = 0.0;

    size_t size() const { return segments.size(); }
    bool empty() const { return segments.empty(); }
    const Segment& at(size_t index) const { return segments.at(index); }
};

class SegmentGraph {
public:
    SegmentGraph() = default;

    // YAML dosyasından haritayı yükler ve parseler
    bool loadFromYAML(const std::string& filepath);

    // Verilen sıralı waypoint noktaları arasında Dijkstra ile rota planlar
    Route planRoute(const std::vector<std::string>& waypoint_node_ids) const;

    // Koordinata en yakın graf düğümünü bulur
    std::string findNearestNode(double x, double y) const;

private:
    std::unordered_map<std::string, GraphNode> nodes_;
    std::vector<Segment> segments_;
    std::unordered_map<std::string, std::vector<size_t>> adjacency_; // Komşuluk
    listesi

    std::vector<Segment> dijkstra(const std::string& start, const std::string& goal)
    const;
    void computeSegmentCosts();
    // ... yardımcı parser metotları (trim, extractDouble, extractString) ...
};

} // namespace robotaxi_bt
#endif

```

Mimari Analiz ve Algoritma Akışı:

1. Veri Yapıları:

- **GraphNode** : Düğümlerin 2B harita üzerindeki (x,y) koordinatlarını tutar.
- **Segment** : Yönlü kenardır. Yolun tipini (tünel, kavşak vb.), şerit bilgisini ve hedef pozisyonu (goal_yaw dahil) saklar.

2. Dijkstra Algoritması (dijkstra metodu):

- Başlangıç düğümünden hedef düğüme giden en kısa yolu bulmak için standart bir öncelikli kuyruk (min-heap priority queue) tabanlı Dijkstra algoritması koşturur.

- Her segmentin maliyeti (`cost`), iki düğüm arasındaki Öklidyen koordinat mesafesi (`std::hypot`) olarak hesaplanır.

3. **planRoute** :

- Robotun uğraması gereken görev noktaları (örneğin: [`"N_START"`, `"N_DURAK_1"`, `"N_PARK_IN"`]) sırayla verilir.
- Her iki sıralı nokta çifti için Dijkstra çağrılarak alt rotalar hesaplanır ve uç uca eklenerek tam rota (`Route`) oluşturulur.